Sideline — Improving Current Features

A product & engineering review Focus: getting people to *use it like an app*, turn on notifications, and come back to check stats & scores. Weighted across UX polish, engagement, admin time, insights, onboarding, and reliability.

How to read this

Each item has an **Impact** (value to your group) and **Effort** (build cost) rating, plus a **Challenge** note where I argue against the obvious version of the idea so the recommendation is the *considered* one, not the first one.

Priority is roughly Impact ÷ Effort. The top three are the ones I'd ship first because they attack the stated pain directly: **adoption, notifications, and habit.**

#	Improvement	Impact	Effort	Priority
1	Make it genuinely installable (real PWA icons + install prompt)	High	Low	**P0**
2	Stop permanently dismissing the notification prompt	High	Low	**P0**
3	Post-match "results are in" + rating-change notifications	High	Med	**P0**
4	Show rank movement & score change on the leaderboard	High	Low	**P1**
5	Cache/snapshot scores instead of recomputing every page load	Med	Med	**P1**
6	Bring the "how scores work" guide inside the app	Med	Low	**P1**
7	App shortcuts & richer manifest (deep links)	Med	Low	**P2**
8	Batch & trusted-player fast-paths for admin approvals	Med	Med	**P2**
9	Availability waitlist auto-promotion	Med	Med	**P2**
10	Light-touch UX consistency pass	Low	Low	**P3**

P0 — Adoption & the notification habit loop

1. Make Sideline genuinely installable

What's there now. `src/app/manifest.ts` ships a single icon — an SVG (`/logo-mark.svg`) with `sizes: "any"` . There is no `beforeinstallprompt` handling anywhere; the only install guidance is the iOS instructions in `src/components/notification-prompt.tsx` .

Why it matters. This is the root of "I want people to use it as an app." Two concrete failures:

- **Android/Chrome** only shows the "Add to Home Screen" / install banner when the manifest has proper raster icons (typically 192×192 and 512×512 PNG, including a `maskable` one). An SVG with `sizes:"any"` frequently fails the installability criteria, so the browser never offers to install — users stay on a browser tab and forget it exists.
- **iOS** ignores SVG for the Home Screen icon and needs a 180×180 `apple-touch-icon`; otherwise the installed icon looks broken, which kills the "feels like an app" impression.

How.

- Add PNG icons at 192, 512, and a 512 `maskable` variant; add a 180×180 `apple-touch-icon`. Reference them in `manifest.ts`.
- Add an **"Install app"** affordance: capture `beforeinstallprompt` in a small client component, stash the event, and show an "Install Sideline" button (Android/desktop). Keep the existing iOS instructions for Safari.
- Set the manifest `theme_color` / `background_color` to match the actual theme (currently hardcoded `#0D0D0D` even in light mode).

Challenge. The lazy version is "just add a bigger icon." That alone won't move the needle — the missing `beforeinstallprompt` button is what turns a passive manifest into an active prompt. But equally, don't build a heavy custom install modal: a single unobtrusive button that appears only when the browser fires the event (i.e., only when install is actually possible) avoids nagging users who can't or already did install.

---

2. Stop permanently silencing the notification prompt

What's there now. In `notification-prompt.tsx`, tapping **"Not now"** writes `localStorage["sideline_push_prompt_dismissed"] = "1"` — **forever**. The prompt also only lives on the dashboard.

Why it matters. Notifications are your re-engagement engine, and this is the single biggest self-inflicted wound against the "enable the notifications" goal. A player who taps "Not now" once (very common on first visit, before they care) is never asked again on that device. You're permanently opting out your most casual users — exactly the ones who need reminding.

How.

- Replace the permanent flag with a **snooze**: store a timestamp and re-surface after, say, 7 days, and only up to N times.
- Re-ask around a **high-intent moment** — e.g., right after a player marks themselves "in" for a match ("Want a heads-up when teams are posted?"). Contextual asks convert far better than a cold dashboard banner.
- Track opt-in state server-side too (you already have push subscriptions), so the decision follows the user across devices rather than living only in one device's `localStorage`.

Challenge. The tempting fix is "just ask again every visit." That's how you train people to reflexively dismiss and, on iOS/Android, risks a hard `denied` you can never recover from in-app. Snooze + contextual re-ask respects the user while still recovering the accidental "Not now."

3. Post-match "results are in" and rating-change notifications

What's there now. From `src/lib/notification-types.ts`, triggers are event-driven and *forward-looking*: match created, match live, availability open, payment due, stats approved, new player, guest request/decision, announcement. There is **nothing** that fires *after* a match to pull people back.

Why it matters. This is the habit loop you described ("stats and scores regular checking"). People don't return on their own to check a leaderboard — they return when something tells them their standing changed. The most powerful, lowest-friction re-engagement message in any competitive app is *"your result is in."* You already compute everything needed (`getPlayerScores`, `getLeaderboard`, score breakdowns); you're just not notifying on it.

How. Add two triggers:

- **"Results are in"** when a matchday moves to `closed` (or when voting closes): *"Ratings are in for Tuesday — see how you did."* Deep-link to the leaderboard or the player's breakdown.
- **"Your rating changed"** (personalized): *"Your rating went 71.2 → 73.8 (+2.6). You moved up 2 places."* Compute the delta against the `legacy_score_snapshots` table (which already exists) captured at the previous match close.

Challenge. Two traps here. First, *over-notifying*: firing a rating-change push to all 30 players after every micro-recompute would be spam — gate it to once per matchday close, and only when the delta is meaningful (e.g., $|\Delta| \geq 1$ or a rank change). Second, *negative framing*: "you dropped 3 places" can demotivate casual players. Prefer neutral/positive framing and let users who want the full picture tap through. Make this type `userConfigurable` so the competitive crowd keeps it and the sensitive crowd can mute it.

P1 — Turning scores into something people watch

4. Show rank movement and score change on the leaderboard

What's there now. `src/app/leaderboard/leaderboard-table.tsx` renders a static ranking. The dashboard shows "Your standing #X of Y" but no sense of *change*.

Why it matters. Movement is what makes a table addictive. A green ▲2 / red ▼1 next to each player, and a "since last match" score delta, turns a lookup into a story. It pairs perfectly with #3 (the notification says "you moved up" → they tap → they *see* the ▲).

How. Use `legacy_score_snapshots` (already in the schema) to store a snapshot per matchday close, then diff current vs previous for a movement column and a small sparkline of the last N scores.

Challenge. Don't over-engineer the snapshot cadence. Snapshot on match close, not on every settings change, or "movement" becomes noise driven by admin tweaks rather than actual play. And hide movement for `unrated` / `provisional` players — a newcomer

swinging 50→64 as their first votes land isn't "form," and showing it as a huge ▲ is misleading.

---

5. Cache/snapshot scores instead of recomputing on every load

What's there now. Both `src/app/page.tsx` and `src/app/leaderboard/page.tsx` are `export const dynamic = "force-dynamic"`, and the dashboard computes the **entire** leaderboard just to find one player's rank.

Why it matters. `getPlayerScores()` runs several queries and a full recompute (votes, outlier filtering, stats, blend) for *every* player on *every* dashboard and leaderboard view. For your group today it's fine; as usage grows (the goal!) this becomes the hot path, and on serverless Postgres every view is billable query load and latency. It also makes the app feel sluggish precisely when engagement is highest (right after a match, everyone checking at once).

How.

- Compute scores once per relevant change (match close, stat approval, vote edit, settings change) and cache them — either Next's `revalidateTag` / `unstable_cache` keyed to a "scores dirty" tag, or persist the computed leaderboard into `legacy_score_snapshots` /a `current_scores` cache table.
- On the dashboard, fetch just the one player's cached rank instead of building the whole board.

Challenge. The instinct is "add caching everywhere." But scores are the product's source of truth — a stale leaderboard right after a correction erodes trust. The right model is *event-driven invalidation* (recompute on the handful of write actions), not a blunt time-based TTL. That keeps it correct and fast. This also directly enables #4 and #3 (snapshots are the diff source), so it's a shared foundation, not isolated plumbing.

---

6. Bring the "how scores work" guide into the app

What's there now. The guide is served as a static file (`public/how-scores-work.html`) and opens in a new browser tab from the leaderboard.

Why it matters. It's the one place you explain the model, but it currently breaks the app shell — no nav, no dark-mode parity, opens outside the PWA (on iOS this can bounce users out of the installed app into Safari, the opposite of the adoption goal).

How. Render the same content as an in-app route (e.g., `/leaderboard/how-scoring-works`) reusing the app layout, or as an expandable panel on the leaderboard. Keep the standalone HTML/PDF as the shareable/exportable artifact.

Challenge. Don't duplicate the content in two hand-maintained files that drift. Make the in-app page the source and generate the exportable PDF from it, or vice-versa — one source, one render path.

P2 — Saving admins time & smoothing edges

7. App shortcuts and deep links

Add manifest `shortcuts` (e.g., long-press the icon → "Set availability", "Leaderboard", "Vote") and ensure notifications deep-link to the exact page. Cheap, and every shortcut is a re-entry point into the habit loop. **Challenge:** only add shortcuts that are useful most days — a shortcut to a rarely-used admin page is clutter.

8. Batch and trusted-player fast-paths for approvals

You have *five* approval queues: stats submissions, goals claims, guest requests, password resets, award overrides (see the `*_audit` and `status` columns across the schema). Each is one-at-a-time. Add **select-all / batch approve**, and an optional **"auto-approve stats from trusted players"** setting so veterans' self-reported goals don't need babysitting. **Challenge:** auto-approval trades integrity for speed — scope it to a per-player "trusted" flag an admin sets deliberately, keep the audit trail, and never auto-approve anything that affects payments.

9. Availability waitlist auto-promotion

`matches.maxPlayers` caps the roster. When the match is full and someone drops out, promoting the next waitlisted player should be automatic (with a push: "A spot opened — you're in"). Today that's presumably manual. **Challenge:** auto-promotion needs a fairness rule (first-in vs skill-balanced) and a confirmation window so a promoted player who can't make it can decline without leaving a hole — otherwise you've just created a silent no-show.

P3 — Consistency & polish

10. Light-touch UX pass

- **Theme color**: `manifest.ts` and `viewport.themeColor` are hardcoded dark; make them respond to the user's theme so the status bar/PWA chrome matches.
- **Empty states**: you already have an `EmptyState` component — make sure every list (no matches, no votes yet, no stats) uses it with a clear next action, so new/quiet periods don't look broken.
- **Dashboard hierarchy**: the home page stacks several cards of similar weight. Lead with the single most important action (availability if open, else "results are in") and demote the rest.
- **Consistent "since last seen" cues**: a subtle dot on nav items (Leaderboard, Matches) when there's something new pulls people in without a notification.

Challenge. Polish is easy to over-invest in. None of these should precede P0/P1 — they're the finishing coat once the adoption and habit mechanics are working, not a substitute for them.

What I deliberately left out (and why)

- **A full design-system overhaul / component library swap.** The existing Tailwind + small `ui/` set is coherent and not the bottleneck; rewriting it would burn effort that #1–#3 need.
- **Real-time (websockets) leaderboard updates.** Tempting, but for a group that plays weekly, event-driven notifications + cached scores deliver the same felt value at a fraction of the complexity and cost.
- **Aggressive re-notification / daily digests.** For a weekly-cadence group this would train people to mute. The match-anchored triggers in #3 match the real rhythm of play.